Aran Learn 2: Indian Identifiers and Check Digits

For: everyone. **Time:** about 5 hours with practice. **Before this:** Learn 1, sections 4.1 to 4.5.

Aran's main idea rests on one fact: Aadhaar and card numbers contain a built-in maths check. This doc teaches that check from zero, by hand first and then in Python, and shows how Aran turns it into a security guarantee.

1. The identifiers Aran protects

A detector can only find what it understands. For each identifier you need to know three things: its **shape** (how to spot a candidate), its **check** (how to prove a candidate is real) and its **harm** (what an attacker can do with it).

Identifier	Issued by	Shape	Example shape	Built-in check	Harm if leaked
Aadhaar	UIDAI	12 digits, first digit 2 to 9	2XXX XXXX XXXX	Verhoeff (last digit)	Identity fraud, SIM swap, fake KYC
Aadhaar Virtual ID (VID)	UIDAI, made by the holder	16 digits	XXXX XXXX XXXX XXXX	Verhoeff	Same as Aadhaar while it is active
Payment card number (PAN in card jargon)	Card networks: RuPay, Visa, Mastercard	13 to 19 digits, usually 16	6XXX XXXX XXXX XXXX	Luhn (last digit)	Card-not-present fraud
PAN (Permanent Account Number)	Income Tax Department	5 letters, 4 digits, 1 letter	ABCPD1234E	A check letter, method not public	Tax fraud, loan fraud
Bank account number	Each bank	9 to 18 digits, bank-specific	varies	None that is public	Targeted phishing, mandate fraud
IFSC	RBI	4 letters, a zero, 6 letters or digits	SBIN0001234	None	Not secret alone, but joins an account to a branch
UPI ID (VPA)	NPCI and banks	name@handle, handle from a fixed list	ramesh@oksbi	None	Collect-request scams
OTP	The bank's OTP service	4 to 8 random digits, valid for minutes	482913	None	Immediate account takeover
Mobile number	Telecom operators	10 digits, first digit 6 to 9	9XXXXXXXXX	None	Phishing, SIM swap target

Anatomy of a card number. The first 6 or 8 digits are the IIN (issuer identification number, also called BIN): they name the network and the bank. The last digit is the Luhn check digit. Everything in between is the account part. So in a 16-digit card with an 8-digit IIN, only 7 digits are really secret. Learn 4 shows why that matters for masking.

Network	Starts with
Visa	4
Mastercard	51 to 55, and 2221 to 2720
RuPay	60, 65, 81, 82, 508

Anatomy of a PAN. `ABCPD1234E`: the first three letters are a sequence, the fourth letter is the holder type (P person, C company, H Hindu undivided family, F firm, T trust, and a few more), the fifth is the first letter of the surname or entity name, then four digits, then a check letter. A detector can use the fourth letter as a sanity check: a PAN with a fourth letter outside the known set is not a PAN.

The rule for the whole project. Never put a real person's identifier in code, tests, slides or chat. Generate numbers that pass the check, as Aran's tests do. Every Aadhaar-shaped number in these docs, such as 2341 2341 2346, was generated and belongs to nobody.

Masking you see every day. A masked Aadhaar shows only the last 4 digits (XXXX XXXX 2346). Card receipts usually show the first 6 and last 4. These look safe, but Learn 4 computes exactly how much they still give away.

2. Modular arithmetic in five minutes

Both checks below use "clock arithmetic". On a clock, 3 hours after 11 o'clock is 2 o'clock, not 14. We say 14 mod 12 = 2: the remainder after dividing by 12.

Expression	Value	Why
17 mod 10	7	17 = 1 × 10 + 7
70 mod 10	0	divides exactly
23 mod 8	7	23 = 2 × 8 + 7
(9 + 4) mod 10	3	only the last digit survives

In Python, `mod` is written `%`: `70 % 10` gives `0`. "The total ends in 0" is the same as "total mod 10 = 0".

3. The Luhn check (card numbers)

3.1 The rule

Hans Peter Luhn at IBM designed this check in 1954 to catch typing errors.

1. Start at the rightmost digit (the check digit) and call it position 0.
2. Double the digit at every odd position (1, 3, 5, ...).

3. If doubling gives more than 9, subtract 9 (the same as adding the two digits: 16 gives 1 + 6 = 7).
4. Add all the values. The number is valid when the total mod 10 is 0.

3.2 Worked example: checking a number

The standard test number 79927398713, read right to left:

Position from right	0	1	2	3	4	5	6	7	8	9	10
Digit	3	1	7	8	9	3	7	2	9	9	7
Double it?	no	yes	no	yes	no	yes	no	yes	no	yes	no
Value used	3	2	7	16 − 9 = 7	9	6	7	4	9	18 − 9 = 9	7

Total = 3 + 2 + 7 + 7 + 9 + 6 + 7 + 4 + 9 + 9 + 7 = 70, and 70 mod 10 = 0, so the number is valid.

3.3 Worked example: computing a check digit

To issue a new number, the bank picks the payload and then computes the one check digit that makes the total end in 0. For the payload 7992739871, try each digit d at the end and keep the one that passes. Leaving out the 3 in the table above, the other values sum to 67, so d must make 67 + d end in 0: d = 3. Python does the same search:

```
from aran_core import luhn_check_digit, luhn_valid
luhn_check_digit("7992739871")          # '3'
luhn_check_digit("411111111111111")     # '1'  -> 4111 1111 1111 1111
```

3.4 What Luhn catches and what it misses

```
luhn_valid("4111111111111111")    # True
luhn_valid("4111111111111121")    # False: one digit changed, caught
luhn_valid("12345096")            # True
luhn_valid("12345906")            # True: 09 swapped to 90, NOT caught
```

Luhn catches every single wrong digit and most swaps of neighbouring digits, but it misses the swap 09 ↔ 90, because 0 and 9 give the same value whether doubled or not (0 → 0, 9 → 18 − 9 = 9). That weakness is why UIDAI chose a stronger check for Aadhaar.

3.5 The Python, line by line

```
        def luhn_valid(digits: str) -> bool:
            total = 0
            for i, ch in enumerate(reversed(digits)):   # i = position from the right
                d = ord(ch) - 48          # turns '7' into 7 (48 is the code of '0')
                if i % 2 == 1:            # every second digit
                    d *= 2
                    if d > 9:
                        d -= 9
                total += d
            return total % 10 == 0
```

This is exactly the function in `research/aran_core.py`. Compare each line with the four rules in 3.1.

4. The Verhoeff check (Aadhaar)

4.1 The idea: digits as moves of a pentagon

In 1969 Jacobus Verhoeff wanted a check that catches every single wrong digit and every swap of neighbours. He found it in group theory, the maths of symmetry.

Take a regular pentagon. There are exactly 10 ways to move it so it lands back on its own outline: 5 rotations (by 0°, 72°, 144°, 216°, 288°) and 5 flips (across each of its 5 lines of symmetry). These 10 moves form the **dihedral group D5**. Verhoeff gave each move a digit: 0 to 4 are the rotations, 5 to 9 are the flips.

"Multiplying" two digits means doing one move and then the other. The result is always one of the 10 moves, so it is again a digit. The crucial property is that **order matters**: rotating then flipping is not the same as flipping then rotating.

```
        from aran_core import _D
        _D[1][5]    # 6  (move 1, then move 5)
        _D[5][1]    # 9  (move 5, then move 1): a different result
```

Luhn uses addition, where 0 + 9 equals 9 + 0, so it cannot always notice a swap. D5 multiplication is not like that, and Verhoeff added a position-based shuffle on top, so swapping any two neighbouring digits always changes the result.

4.2 The three tables

The algorithm uses three small tables, all listed at the top of `research/aran_core.py`:

- **d** (`_D`): the multiplication table of D5, 10 × 10. `d(a, b)` = "do move a, then move b".
- **p** (`_P`): 8 rows of shuffles. Row k says how to relabel a digit at a position whose number mod 8 is k. It is what makes position matter.
- **inv** (`_INV`): for each move, the move that undoes it. Used only when computing a check digit.

4.3 Checking a number

Read the digits right to left. Keep a running value c, starting at 0. At position i:

```
c = d( c, p(i mod 8, digit_i) )
```

The number is valid if c = 0 at the end.

Worked example with the short number 2363:

Position i (from right)	Digit	p(i mod 8, digit)	c before	c after = d(c, p)
0	3	p(0, 3) = 3	0	d(0, 3) = 3
1	6	p(1, 6) = 3	3	d(3, 3) = 1
2	3	p(2, 3) = 3	1	d(1, 3) = 4
3	2	p(3, 2) = 1	4	d(4, 1) = 0

The final c is 0, so 2363 is valid. Look up each value yourself in `_P` and `_D` (row first, then column, counting from 0) to make sure you can follow it.

4.4 Computing a check digit

To find the check digit for a payload, the payload digits sit one place further left than they will in the final number, so use positions starting at 1. Then take the inverse of the final c.

Worked example for the payload 236:

Position (i + 1)	Digit	p	c before	c after
1	6	p(1, 6) = 3	0	3
2	3	p(2, 3) = 3	3	1
3	2	p(3, 2) = 1	1	2

Final c = 2, and inv(2) = 3, so the check digit is 3 and the full number is 2363, which matches 4.3.

```
        from aran_core import verhoeff_check_digit, verhoeff_valid
        verhoeff_check_digit("236")            # '3'
        verhoeff_check_digit("23412341234")    # '6'  -> 2341 2341 2346
        verhoeff_valid("234123412346")         # True
```

4.5 What Verhoeff catches that Luhn misses

```
        verhoeff_valid("23412345093")    # True  (a valid generated number ending ...093)
        verhoeff_valid("23412345903")    # False (09 swapped to 90): caught
```

Error type	Luhn	Verhoeff
One wrong digit	Always caught	Always caught
Two neighbours swapped	Caught except 09 ↔ 90	Always caught
Random 12 or 16 digits passing by chance	1 in 10	1 in 10

5. Two facts Aran builds on

Fact 1: for any payload, exactly one check digit passes and the other nine fail. This follows from "every single wrong digit is caught": if two different last digits both passed, changing one into the other would be an undetected single-digit error.

Fact 2: random digits pass only 1 time in 10. Generate 1,000 random 12-digit strings and about 100 pass Verhoeff (a run with seed 1 gave 105). This is why the checksum alone cut false alarms on real logs from 366 to 4, and the remaining 4 are the 1-in-10 chance at work.

```
        import random
        r = random.Random(1)
        sum(verhoeff_valid("".join(r.choice("0123456789") for _ in range(12)))
            for _ in range(1000))       # 105
```

6. How Aran turns the check into a security guarantee

6.1 Checksum-complement tokens

A token is a stand-in value that systems store instead of the real identifier. Aran's tokens keep the 12-digit shape, so databases and forms still accept them, but they deliberately carry a **wrong** check digit.

```
        def cc_token_aadhaar(ff1, key, aadhaar, sink):
            body = ff1.encrypt(aadhaar[:11], sink.encode())          # 11 scrambled digits
(Learn 3)
            bad = (int(verhoeff_check_digit(body)) + _offset(key, sink)) % 10
            return body + str(bad)                                   # offset is 1 to 9,
never 0
```

Because the offset is never 0, `bad` is never the correct check digit, and by Fact 1 the token always fails Verhoeff. This is Theorem 1 in the research doc (perfect separability): real Aadhaar numbers always pass, Aran tokens never pass.

6.2 Worked example

Using demo keys (all-zero FF1 key, and `b"k" * 32` for the offset key; real keys live in an HSM), the same invented Aadhaar 2341 2341 2346 becomes a different token for each destination ("sink"):

Sink	FF1 body (11 digits)	Correct check digit	Offset	Token	Passes Verhoeff?
crm	55258817336	1	1	552588173362	No
logs	78936098212	0	6	789360982126	No
analytics	23487493164	3	8	234874931641	No

6.3 Getting the real value back

An authorised system decrypts the 11 body digits with the same sink name and recomputes the correct check digit, so nothing is lost:

```
        cc_detokenize_aadhaar(ff1, "552588173362", "crm")    # '234123412346'
```

6.4 What this gives the bank

- **A scanner can prove what it found.** If a 12-digit number in a log passes Verhoeff, it is either a real Aadhaar or a 1-in-10 coincidence, never an Aran token. If it fails, it is not a real Aadhaar.
- **A leaked token names its source.** Because each sink has its own tokens (Theorem 2), finding `789360982126` in a dump on the internet says it came from the logs system.
- **The same idea works for cards** with Luhn instead of Verhoeff.

7. Practice

- **P1.** By hand, check 4111 1111 1111 1111 with Luhn. Then confirm with `luhn_valid`.
- **P2.** By hand, compute the Verhoeff check digit for the payload 41 using `_P` and `_D`. Confirm with `verhoeff_check_digit("41")`.
- **P3.** Generate 1,000 random 12-digit strings and count how many pass `verhoeff_valid`. Repeat with 10,000. Does the share get closer to 10%?
- **P4.** Write a function that tries every swap of neighbouring digits in a valid 12-digit number and reports which swaps Luhn misses and which Verhoeff misses.
- **P5.** Write `looks_like_pan(s)` using a regex and the fourth-letter rule. Test it on `ABCPD1234E` (yes), `ABCXD1234E` (no) and `ABCPD12345` (no).
- **P6.** Read `cc_token_aadhaar` in `research/aran_core.py` and explain to a teammate, in three sentences, why its output can never pass Verhoeff.

Hints. P2: start with c = 0, the digit 1 is at position 1 and 4 at position 2; take `_INV` of the final c. P4: build the swapped string with `s[:i] + s[i+1] + s[i] + s[i+2:]`.

8. Further reading

- "Luhn algorithm" and "Verhoeff algorithm" on Wikipedia, which list the full tables.
- J. Verhoeff, "Error Detecting Decimal Codes" (1969), the original.
- Dihedral groups: Gallian, "Contemporary Abstract Algebra", chapter 1, or the free "Visual Group Theory" lectures by Nathan Carter.
- The Aran research doc, section on Theorem 1.